WordPress Security Audit & Auto-Remediation Suite

Comprehensive Operational Guide & Architecture Documentation (v4.0 / v2.1)

1. Executive Overview

The **WordPress Security Audit & Auto-Remediation Suite** consists of two shell scripts designed to run natively within Linux server environments hosting WordPress installations. The pair operates in a closed-loop security workflow: `scanner.sh` performs enterprise-grade vulnerability scanning and risk assessment, while `remediador.sh` automatically ingests the scanner's output report and applies hardening fixes to eliminate identified security flaws.

2. System Components & Architecture

2.1 Scanner Script (`scanner.sh` v4.0)

The auditing agent conducts non-destructive static and dynamic checks across six distinct vectors of the target WordPress directory:

- **Core Integrity Check:** Queries the official `api.wordpress.org` REST endpoint to fetch checksum hashes (MD5) matching the exact installed WordPress version and locale. It verifies all core files against these official signatures to detect tampered core code.
- **Malware & Webshell Scan:** Scans the `wp-content/` directory for high-risk structural regex patterns, such as obfuscated execution constructs (e.g., `eval(base64_decode)`, `gzinflate`, `str_rot13`, and backdoor superglobal invocations).
- **Uploads & Hidden Files Inspection:** Detects malicious executable `.php` scripts disguised inside the `wp-content/uploads/` media directory and audits unexpected dot-files across the web root.
- **Permissions & Linux Hardening Audit:** Identifies dangerous global permissions (CHMOD 777) on directories and overly permissive settings on `wp-config.php`.
- **WP Configuration Hardening Audit:** Checks if file editing via the WP Admin dashboard is disabled (`DISALLOW_FILE_EDIT`) and verifies whether default, unsecure salt keys are still present in `wp-config.php`.
- **Database Script Injection Inspection:** Utilizes `WP-CLI` (if available) to scan the `wp_options` table for inline cross-site scripting (XSS) or malicious `<script>` tags.

2.2 Remediation Agent (`remediador.sh` v2.1)

The auto-healing agent parses the latest security report generated by `scanner.sh` using fuzzy regex patterns. If matching vulnerabilities are identified, it applies targeted corrective actions:

- **System Cleanup:** Purges temporary and non-essential dot-files (e.g., macOS `.DS_Store` files).
- **Permission Remediation:** Restricts `wp-config.php` permissions to `640` and resets dangerous `777` directory permissions back to the secure standard `755`.
- **Dashboard Hardening:** Injects `define('DISALLOW_FILE_EDIT', true);` into `wp-config.php` to prevent dashboard file modification attacks.

- **Uploads Quarantine & Isolation:** Appends a `.quarentena` extension to PHP files found in `wp-content/uploads/` and automatically creates a restrictive `.htaccess` file denying script execution in the uploads directory.
- **Core File Restoration:** Re-downloads and overwrites corrupted core files using `WP-CLI` (when configured).

3. Prerequisites & Environment Requirements

To run both scripts smoothly, your host environment must satisfy the following dependencies:

Requirement / Tool	Status	Purpose
Linux OS / Bash Shell	Mandatory	Bash 4.0+ with basic GNU tools (`find` , `grep` , `sed` , `stat` , `chmod`).
cURL & md5sum	Mandatory	Required for fetching WordPress official checksums and verifying hashes.
WordPress Installation	Mandatory	Scripts must be run directly from the root directory of the WP installation.
WP-CLI	Optional	Enables DB script injection checks and automatic WP core restoration.

4. Step-by-Step Installation & Usage Guide

Step 1: Deployment & Placement

Upload `scanner.sh` and `remediador.sh` directly to the root directory of your WordPress website (the folder containing `wp-config.php`):

```
cd /var/www/html/my-wordpress-site/
```

Step 2: Assign Execution Permissions

Grant execution rights to both scripts using `chmod` :

```
chmod +x scanner.sh remediador.sh
```

Step 3: Execute Security Audit (Scanner)

Run the scanner to perform the complete security audit and generate a timestamped report file (e.g., `relatorio_seguranca_YYYY-MM-DD_HH-MM-SS.txt`):

```
./scanner.sh
```

Step 4: Execute Auto-Remediation Agent

Run the remediator script. It automatically locates the latest report file and prompts for confirmation before applying fixes:

```
./remediador.sh
```

> **IMPORTANT OPERATIONAL NOTE**
>
> After running the remediator, always re-run `./scanner.sh` to re-evaluate the environment and confirm that your security score has reached 100/100.

5. Workflow Cycle Matrix

Phase	Executed Command	Input File	Generated Output
1. Audit	`./scanner.sh`	WordPress Filesystem / DB	`relatorio_seguranca_TIMESTAMP.txt`
2. Mitigation	`./remediador.sh`	Latest `relatorio_seguranca_*.txt`	Hardened WP Environment
3. Validation	`./scanner.sh`	Post-remediation Filesystem	Updated Report (Target: 100/100)